One Language to Hear: A Translation Gateway for Live Phone Calls

Sharad Dixit
Birla Institute of Technology and Science, Pilani
Rajasthan, India
dixitsharad@outlook.com

Pritee Parwekar
Dept. of Computer Science and System Engineering
GITAM School of Computer Science Engineering
GITAM (Deemed to be University)
Hyderabad, India
pparweka@gitam.edu

***Abstract*—People who do not share a language have difficulty on a phone call. A direct audio path carries the speaker's original voice, so a translated call has to replace it with generated speech in a language the listener understands. This paper describes a calling system in which each person chooses the language they want to hear and read on their own device. A server-side gateway recognizes each person's speech, translates it into the other person's chosen language, and plays it with a matching neural voice. Either person can change language during the call: changing the other person's language only updates the translation target and voice, while changing one's own language restarts that person's recognizer. Indian target languages are returned in native script. Media passes as 20 ms PCM frames through streaming recognition, translation and synthesis, and a playback-time gate mutes a phone's microphone while translated speech is still queued on it. We measured the implemented system on synthetic speech in four languages on one machine. Recognition restricted to the two languages of the call (word error rate 9.6–12.1%) or set to the speaker's own language (9.2–11.7%) was far more accurate than a ten-language candidate list (42.1–43.3%). The median time from the end of speech to the first translated audio was 1.43 s, about twice the 0.7 s design budget. In a calculation on measured chunk timings, a gate that follows the playback queue covered all of the playback, against 10–17% for a per-chunk gate. We state what these results do and do not show.**

***Index Terms*—speech-to-speech translation, live calling, language identification, preferred language, Indic script, streaming cascade.**

I. Introduction

So if we're having a conversation over the phone, then there will be a very small gap between the time you speak and the time another person responds [1]. If it takes more than 150ms for the sound to go from one side to the other, then it degrades the quality of the conversation, and people have to adapt their timing [2]. And when you factor in the time taken for speech recognition, translation, and speech synthesis, it gets even worse and the whole thing becomes super awkward.

So most apps force you to choose what language you're speaking in and what language the other person is listening in, but this doesn't work in a real-life conversation. A person might reply to you in Tamil, English, or some mix of both, but they might still want all of what they hear translated to Tamil. At the same time, the person on the other end might want all of what they hear to be in Hindi. This means the best approach is to translate the speaker's words to whatever the listener wants to hear. The first version of the system used the speaker's language preference only as a hint when detecting which language was being spoken. The measurements in Section X show that using the language the person has chosen, or restricting detection to the languages of the call, works much better.

In this system each user sets a language in their profile. It can be changed before a call or at any time during it. The gateway recognizes each person's speech and translates it for the listener, and the audio is synthesized in the listener's language. If the speaker and the listener already speak the same language, no translation is needed and the text is spoken as recognized. The client is written in Flutter, signalling is handled by a Node.js service, and media is handled by a Python service. Account verification is part of the product and is not the subject of this paper. The paper makes three contributions. First, it describes a calling design built around the language each person chooses to hear, with changes allowed in mid-call. Second, it reports how the design performed when measured: recognition, translation, latency, mid-call language change and echo control (Section X). Third, it reports where the earlier version of the design was wrong. The original recognizer filled a ten-language candidate list around the two preferred languages. In our measurements that list gave about four times the word error rate of recognition restricted to the languages of the call, and the deployed configuration no longer uses it.

II. Related Work

The Direct Speech-to-Speech models map directly from source audio to target audio without producing a text transcription. Translatotron is a direct speech-to-speech model. It is a sequence-to-sequence model that does not produce a text output [3]. Translatotron 2 modifies this approach by allowing it to maintain the quality of the original speaker's voice [4]. SeamlessM4T puts multiple languages into one model that takes audio and text inputs and generates audio and text outputs [5]. If the question is about quality or preserving the voice of the source speaker, these are relevant baselines. This paper is not about training the model. Since the cascaded interpreter recognizes speech, translates text, and synthesizes voices, we use a cascade of these services. When a user makes a phone call, the person receiving the call has to

hear something, speak in his own language, have captions, and have a good answer for echo. Our work would also be able to do all this. In the near future, a well-designed direct speech-to-speech model might replace the cascade, without affecting the functionality of the speech-to-speech session.

Cascaded interpreters recognize speech, translate text, and synthesize voices. The cascade is a waterfall, in which each stage can be replaced, cached, or rate-limited; many errors from one stage cause obvious problems for the next. For instance, a misidentified language causes the translator to translate in the wrong direction, and Romanized text from an Indian language can make a neural voice sound like English. Delaying until the final transcription or until the whole utterance completes adds latency. We do not leave the cascade; we simply focus our efforts on the weak points in the cascade.

Many commercial products already support real-time translation within ordinary phone calls. Some claim to preserve the voice of the source speaker; others use standard text-to-speech voices. The ACL 2026 System Demonstration (WIGVO) describes a server-mediated solution for bidirectional speech translation over PSTN calls, which uses dual-session echo gating to prevent feedback loops from causing translation loops, and also provides latency and translation quality metrics [6]. Our work focuses on continuous recognition, retargeting mid-call without re-starting the recognizer, and using native script for Indian target languages, and differs in scope from their demonstrated system, and so we do not compare metrics quantitatively.

For Indian languages, IndicTrans2 is a suite of models covering all 22 scheduled languages and can serve as a drop-in replacement for the cloud-based translator we use here [7]. The mobile system uses a cloud-based translator that accepts Hindi, Tamil, Telugu, Malayalam, Gujarati, Marathi, and Bengali, as well as Indian English, German, Mandarin, French, Spanish, Dutch, European and Brazilian Portuguese. Corpora such as FLEURS include paired audio and translations in many of these languages, and can be used to evaluate the cascaded system [8]. However, a conversation between two people is inherently more code-mixed and more overlapping than read speech; FLEURS alone cannot measure turn-taking.

Endpointing and language recognition add a few constraints to the system. We use the Silero VAD [9] as our endpointer, which closes the statement after a certain amount of silence (180 ms in our configuration; Silero's own default minimum silence is 100 ms), and caps the maximum duration of the utterance. A lower silence threshold lowers the latency for short statements but may cut off speakers who pause while speaking. Our recognition service accepts a shortlist of candidate languages in automatic mode [10], with at most ten candidates and one locale per base language. An earlier version of this design pinned the two preferred languages first and filled the remaining slots from the catalog. Section X shows that this list was much less accurate than a list holding only the two languages of the call, so the deployed configuration recognizes in the speaker's own language and keeps the two-language automatic mode as an alternative.

III. Hear-Language Calls

A. What Each Person Configures

Everyone has a choice of language that they like best. It is called 'H' for them. In a conversation between A and B there are two listening languages stored in the session, HA and HB. When A talks there is some text (x) which comes out along with information about the language. The listening language is HB. So if the language of x and HB are the same, then B listens to (x). But otherwise what B listens to is determined by the translator according to the target language HB and the desired style. Likewise B's speech is translated into HA. This determines what each person sees on his/her screen. The text that the other person is saying is translated into a language that you understand but the speaker sees the text in his/her language. These two texts are not the same.

B. Why a Directed Pair Is the Wrong Control

For any particular combination of S and D, the correct assignment of roles must be flipped as soon as the other party speaks. Any switch of language by a speaker, either in one word (for instance, a proper name), one digit, or even an entire sentence, will cause the current assignment to be wrong again until the combination is reset. The language each person chooses (hear_lang) is stable: it changes only when that person changes it. For this reason the deployed recognizer uses each person's own chosen language instead of detecting the language of every utterance (Section V-B).

C. Mid-Call Change

Suppose user B changes the language B wants to hear (HB) to a new value. B's phone sends the new language to the signalling service, which passes it to A's media leg, and also sends it to B's own media leg. A's leg changes its translation target and voice to the new value and cancels any synthesis in progress. It does not restart A's recognizer, because A is still speaking the same language. B's own leg, however, now has a speaker who talks in the new language, so it restarts B's recognizer in that language. Utterances from A are translated into B's new language from then on, and B's speech is translated into HA as before. Section X-D measures the second case. When the recognizer was left unchanged after the switch, every test utterance was recognized wrongly; with the restart, the word error rate was 4.5% and 6.4% for two delays after the switch.

IV. Architecture

The translated audio cannot follow the path of the peer-to-peer microphone. The peer connection carrying the microphone input will instead deliver the original language to the receiver. The system treats the server as a translation gateway; each phone sends audio from its microphone to the gateway and receives synthesized audio and captions in return. Signalling—including dialling, ringing, accepting, hanging up, and exchanging call session offers and replies—continues

throughout the call and audio heard on the other side is the synthesized PCM output of the gateway.

Client: A Flutter application that captures 16 kHz, 16-bit, mono PCM audio, while maintaining both signalling sockets and media sockets. At the beginning of the session, the client sends the language its user speaks and chooses to hear (hear_lang) and the other party's listening language (dst_lang); the source language field is sent as "auto" and is not used for recognition, and a partial and final caption is displayed, classified as local or remote. The synthesized PCM audio is processed by a small jitter buffer. The "Primary Language" control allows updating of the listening language via the signalling socket. The client handles registration, login, invitations, and verified updates of phone numbers. Access does not create an account; Registration requires a one-time SMS code and display name. When a phone number is changed, the code sent to the new number is used to verify the update, and the account is transferred accordingly.

Signalling: The Node.js service manages authentication using one-time SMS codes, maintains user records and pending invites within the PostgreSQL database, and monitors user presence.

When the call is initiated, the caller's preferred listening language is picked from his set priority language. The listening language of the call receiver is determined based on their current status or stored preferences; it is not selected by the caller. The values on both sides of the call are stored in the call record. When a client updates its language, the listening language of its client side is updated, and this new language is transmitted to the other client.

Orchestrator: In a Python-based process, each call is a "room," with a separate "translation leg" for each participant. Each leg has a detector, a translator, a synthesizer, a voice-activity detector, and a latency detector. Specific providers are determined by configuration. The deployment described here uses Azure Speech and Translation, along with Google, Sarvam, a small Large Language Model (LLM) translator, and a "mock" pipeline—all accessible via a consistent interface. The fake path is only used to test the wiring of the system; It does not function as an evaluation mechanism.

The signalling component and the orchestrator operate as separate processes. In any study of speech quality, it should be noted that two processes were active along the path of recorded calls. This paper describes the implemented design; It does not present the results of field studies.

V. Media Session

A. Frames

Microphone audio arrives as 16-bit, 16kHz, mono, PCM audio, in 20ms frames. A 640-byte payload constitutes each 20ms frame. Each binary frame has an 8-byte 'little-endian' header with a sequence number, a tag indicating whether it's an uplink microphone audio frame or a downlink synthesized audio frame, an 'end flag', and a reserved field.

Control messages are sent on the same socket, via JSON text frames, for starting sessions, sending text to be spoken, telling the server to start or cancel speech synthesis, ducking and latency reporting.

B. Recognition

Each media leg is started with three language codes: src_lang (always "auto", and no longer used for recognition), hear_lang (the language this person speaks and chooses to hear) and dst_lang (the other person's language). Three recognition configurations exist in the code. In the deployed configuration the recognizer is started in hear_lang and does not detect language at all. In the two-language configuration, automatic detection chooses only between hear_lang and dst_lang. In the filled configuration, which is how the first version of this design worked, those two languages are pinned first and the remaining slots are filled from the catalog up to ten.

In automatic mode the speech service accepts at most ten candidate locales and only one locale per base language, so some catalog languages are absent from the list. With no preferences the list follows catalog order and drops French, Spanish, Dutch and Portuguese. A language that is not in the list is never detected, even if it is in the catalog. Table VIII shows the lists returned for several sets of preferences.

During speech the recognizer produces partial and final transcripts. In the deployed configuration the leg uses hear_lang as the spoken language for every transcript. In automatic mode, the language reported with the result is used instead, and any value other than "auto" or "und" is taken as the language of the speaker.

C. Translation

The translation happens from the final text copies and partial translations start from the interim text, but only after a 40ms settlement delay. Every new interim update increases the internal sequence number, thus causing any preceding interim operations to drop their result. The translation then proceeds with a known spoken language and the current destination. If that combination of spoken language and destination resolves to a matching language code, the text is simply returned unmodified (for instance, if the speaker is English, and the destination is also English, there is no need to translate). Note however that this does not apply to automatic or undetermined source codes. If the source language is automatic, or there is no source language at all, the translator receives an empty string so that it can determine its own source. Be aware that this second language search may sometimes return a different answer from the first selection made by the speech service. Target languages which have a specific script requirement will add that directly to the API request, the list of supported scripts being Devanagari, Tamil, Telugu, Malayalam, Gujarati and Bengali (note that both Marathi and Hindi use Devanagari). If the translator's response contains more than half Latin characters in the result, the leg will flag this and re-request the target language in its original script. The script mapping is consistent with our voice table, as Hindi is mapped to hi-IN-SwaraNeural, Tamil is ta-IN-PallaviNeural, etc. The reasoning here is to ensure that the response comes in the correct script, because otherwise you would have a Hindi

voice reading back text written using the Roman alphabet. While this guarantees correct script, we do not make the claim that this is necessarily the best translation for a given language. Correct script is measured separately.

The in-process cache can hold 2048 source strings. If the final copy is identical to the last interim text that has been translated, the cache returns the archived response, avoiding another trip to the network. The cache is only flushed when it is full, and is purely used for optimizing latency, not for enhancing quality.

D. Synthesis, Cancellation, and Captions

The synthesis process generates a continuous flow of PCM audio segments that correspond to the desired tone. Upon receipt of the first segment, the system marks the time at which the first bytes were received, monitors the delay, and reports the phase to the user. If the user modifies the target language, terminates existing speech, or initiates a new sentence, the generating counter is incremented. All other audio segments linked to the previous generation are discarded, and the app client is notified to halt playback.

Subtitles with audio: Subtitles on the local side show text recognized in the source language, whereas subtitles on the remote side will show target-language translated output. Captions on the remote side could be shown even if the final transcript is not yet produced (this is a partial translation), though we need to assess how readable such early versions are.

VI. Latency as a Design Budget

Table I describes the latency budget, which affects decisions like frame size, silence threshold, and streaming mode. It's an engineering budget, not data gathered on a live call.

TABLE I
DESIGN ALLOCATION FROM THE END OF DETECTED SPEECH TOWARD THE FIRST SYNTHESIZED BYTE. TARGETS ARE NOT MEASUREMENTS.

Stage	Target	What the implementation does
Capture & packetization	25 ms	16 kHz PCM, 20 ms frames
Uplink	60 ms	Media socket; outside the server probe
Silence endpoint	180 ms	Configured value (Silero default is 100 ms)
First usable recognition	150 ms	Streaming partials; continuous ID in auto mode
Translation	80 ms	Cloud translator, 1.5 s HTTP timeout, short cache
First synthesized byte	150 ms	Streaming PCM; playback starts before synthesis ends
Downlink and jitter	50 ms	Short client buffer; outside the server probe
Sum of these targets	about 700 ms	A budget. Design cap is 1 s

For measuring the inside latency, the server includes an internal monitoring tool that records events starting from the start of the monitor process, such as the end of the voice segment, the beginning of the partial transcription, the end of speech, the transition in translations, and the first sample of synthesized speech. This monitoring tool doesn't cover the whole physical loop, missing when the speaker finishes their last spoken sound and when the synthesized audio actually hits the listener's eardrums. Things like network traffic on the way up and down, jitter buffers, and local hardware delays all happen outside these server timestamps.

As a result, it's only possible to confirm that the inside latency is less than 1 second if the time clocks on the receiving device and the server are in sync. If they are not synchronized, then the inside latency of 1 second from Table I is a goal to achieve, not something we've proven.

Fig. 1 plots the seven stage targets of Table I as bars. Added up they come to 695 ms, which Table I rounds to about 700 ms.

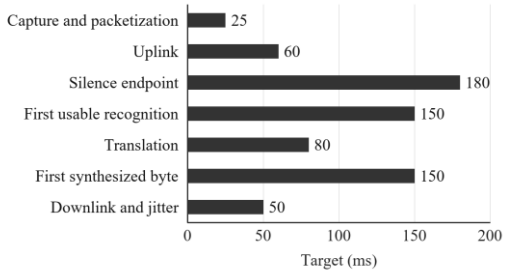


Fig. 1. Per-stage design targets of the cascade from Table I, in the order we used to build the budget. No value here was measured.

The system implements several engineering tricks to stay within the latency budget. Rather than waiting for sentence-ending punctuation, it leverages silent-recognition endpoints to indicate the end of a sentence. The transcription system keeps a queue of sentence-endings that records when vocalization is considered complete, and keeps multiple translation drafts in-flight corresponding to each partial-transcript, which are discarded whenever a new partial-transcript comes in. Finally, it maintains a single, unbroken intro-stream so as not to incur the penalty of opening a new stream each time a user switches languages. Audio output is streamed in real-time as a pre-encoded series of PCM frames, so there is no waiting for the file to exist before it can begin playing. However, we can see that this number is merely a diagnostic metric rather than a true benchmark of the system's performance.

Section X-C measures these stages. From the end of speech to the first translated audio at the listener's socket the median was 1.43 s (95th percentile 2.16 s) when one call ran at a time, about twice the 0.7 s budget in Table I. Recognition finished 0.40 s after the end of speech against 0.33 s budgeted, translation took 0.19 s against 0.08 s, and the first synthesized byte arrived 0.81 s after translation, against 0.15 s. The one-second design cap is therefore not met by this implementation in this setup. Table I is a target, not a description of the measured system.

VII. Echo, Ducking, and Barge-in

Here we introduce a new source of errors which do not occur in non-translational communication. Imagine a Tamil speaker calls a Hindi speaker, with the Hindi speaker's phone on speakerphone. The translation into Hindi is played on the loud speakerphone, but this speech is also fed into the listener's microphone, which the system will recognize as

another instance of speech and translate back into Tamil. This means that the person speaking Tamil receives back an audio feed of herself, delayed by several hundred milliseconds.

To mitigate this, the gateway keeps a guard time for each phone. When translated audio is sent to a phone, the guard runs until everything already queued on that phone has finished playing, plus a fixed 0.35 s tail, and no microphone audio from that phone is passed to recognition during this period. The first version of this gate lasted only for the duration of the latest audio chunk plus the tail, counted from when the chunk arrived. Because synthesis is much faster than playback (16 to 32 times real time in Section X-E), that gate closed less than a second after the voice began and left 83 to 90% of the playback unguarded. The gate is a timer: it does not account for room acoustics, playback volume or Bluetooth latency, and it cuts off the first words of a person who starts speaking while a translation is playing. Two phones in the same room also hear each other. An optional guard, off by default, ignores a speaker's microphone while that speaker's translation is audible on the other phone. It stops the two phones from translating each other's voices in a loop, but it also stops a person from talking over the translation.

Other mechanisms work in the opposite direction. If activated, barge-in detection will cancel all ongoing synthesis as soon as the VAD detects speech coming from the listening side. In addition, the semi-duplex mode provides a secondary layer of protection, since the client device explicitly mutes the other device whenever the client is synthesizing speech. In practice, these mechanisms can be in conflict with each other. Barge-in detection may suppress a user's utterance if they begin to speak too soon after the translation has started. Barge-in can cut off the very part of the translation that the user would prefer to hear. And semi-duplex can cut off very quick interjections such as "ok" or "right". Overlapping speech was not tested in Section X and is listed in Section XI as remaining work.

VIII. Language Inventory

We do not require each client to store local language lists: all the data used for routing languages (recognition codes, translator codes, neural voice labels, and writing scripts) is stored in one central lookup table (Table II). Adding support for a new language merely requires adding a row to that map. In checking whether two participants are speaking the same language, the system compares their translator codes; there are a couple of exceptions requiring manual adjustment: Chinese has different codes for speech recognition (zh-CN) and translation (zh-Hans); English defaults to en-IN (Indian English) for both speech-to-text and text-to-speech; and the two Portuguese dialects use the same recognition code and neural voice label, but use pt-pt and pt (for European Portuguese and Brazilian Portuguese, respectively) as their translation codes.

TABLE II
Catalog Shipped in the Orchestrator. Script Is Requested Only Where a Cell Is Filled.

Language	Speech	Translator	Voice	Script
Hindi	hi-IN	hi	hi-IN-SwaraNeural	Devanagari
Tamil	ta-IN	ta	ta-IN-PallaviNeural	Tamil
Telugu	te-IN	te	te-IN-ShrutiNeural	Telugu
Malayalam	ml-IN	ml	ml-IN-SobhanaNeural	Malayalam
Gujarati	gu-IN	gu	gu-IN-DhwaniNeural	Gujarati
Marathi	mr-IN	mr	mr-IN-AarohiNeural	Devanagari
Bengali	bn-IN	bn	bn-IN-TanishaaNeural	Bengali
English (India)	en-IN	en	en-IN-NeerjaNeural	–
German	de-DE	de	de-DE-KatjaNeural	–
Chinese (Mandarin)	zh-CN	zh-Hans	zh-CN-XiaoxiaoNeural	–
French	fr-FR	fr	fr-FR-DeniseNeural	–
Spanish	es-ES	es	es-ES-ElviraNeural	–
Dutch	nl-NL	nl	nl-NL-ColetteNeural	–
Portuguese (Portugal)	pt-PT	pt-pt	pt-PT-RaquelNeural	–
Portuguese (Brazil)	pt-BR	pt	pt-BR-FranciscaNeural	–

IX. Implementation Status

The software described in this paper includes the following core functionalities. Registration requires a display name and a valid SMS code. Attempting to sign in from an unrecognized number does not automatically create a new account. If a pending invitation has been sent to a phone number that is subsequently registered, the system automatically accepts that invitation upon account creation. Updating a mobile number requires verification of the new number; previously used numbers are blocked, and any pending invitations are automatically linked to the new number.

During call setup, the system records the preferred language (the language in which they wish to listen) for both parties. The caller cannot determine the language of the call recipient; instead, the system relies on the recipient's pre-saved preference, and these roles are never swapped for an incoming call. Once a media session begins, speech recognition runs in the participant's own language.

Changing the other person's language during a call updates only the target language and the voice; changing one's own language restarts that person's recognizer. Active sessions also include several safeguards: shortcuts for same-language translation; the removal of the source field when the input language is unknown; explicit script targeting for text in Latin characters; sequenced partial translations; server-side logging; a guard time that follows the playback queue; half-duplex ducking signals; suspension of playback when a user barges in; and, by default, no forwarding of the speaker's original voice to the listener. An earlier build forwarded the original voice whenever no translation was playing, so the listener could hear a sentence in the speaker's language before its translation. To verify basic socket routing, a small smoke test can be run locally without a cloud certificate, though it does not test translation quality.

X. Results

This section reports measurements of the implemented system, taken on 29 October 2026. All speech was synthetic. Ten English sentences, and their machine translations into

Hindi, Marathi and Tamil, were spoken by Azure neural voices (en-IN, hi-IN, mr-IN and ta-IN) and sent to the system as 16 kHz PCM. This gave 40 clips of 2.3 to 6.1 s (mean 4.0 s), each ending in about 0.8 s of silence. Recognition, translation and synthesis used Azure services in Central India, and the orchestrator ran in a Docker container on one Windows PC. Clean synthetic speech from one voice per language is easier than real calls, so the recognition scores are optimistic and should be read as a comparison between configurations, not as the accuracy to expect for callers. The translation references are machine translations, so no translation score against human references is reported. The recognition experiment was run twice on the same clips, and both runs are shown. Subsections A to E are new measurements. Subsections F to H keep the function-level checks of the earlier version of this paper.

A. Recognition Under Four Configurations

Table III compares four ways of choosing the recognition language for the same 40 clips: the language of the speaker fixed in advance; automatic detection between the two languages of the call; the original list with those two languages pinned first and the remaining slots filled to ten; and a list of ten in catalog order with no preferences. For the two-language configuration the second language was English for Hindi, Marathi and Tamil speech and Hindi for English speech.

TABLE III
WORD ERROR RATE (WER), CHARACTER ERROR RATE (CER) AND LANGUAGE IDENTIFICATION ON 40 SYNTHETIC CLIPS, TWO RUNS

Configuration	WER % run 1	WER % run 2	CER % mean	Language correct, run 1; run 2
Fixed to speaker's language	9.2	11.7	7.7	n/a
Detect: 2 call languages	12.1	9.6	8.3	37/39; 38/40
Detect: 2 pinned + fill to 10	43.3	42.1	34.6	24/39; 24/39
Detect: 10, catalog order	42.5	42.5	34.0	24/39; 24/39

Restricting recognition to the two languages of the call, or fixing the speaker's language, gave a word error rate of 9.2–12.1%. Filling the list to ten languages raised it to 42.1–43.3%, and a list of ten in catalog order gave 42.5% (Fig. 2). Paired across clips and averaged over the two runs, the two-language configuration differed from fixed language by +0.4 points (95% bootstrap interval −3.4 to +4.1), so the two cannot be told apart on this material. The filled list was worse than fixed language by +32.2 points (interval +19.1 to +46.7). The language was identified correctly in 37 of 39 clips (run 1) and 38 of 40 (run 2) with two candidates, and in only 24 of 39 clips with ten, in both runs.

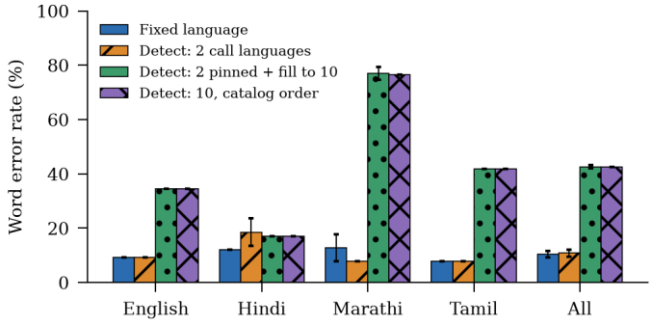


Fig. 2. Word error rate by speaker language for the four recognition configurations. Bars are the mean of two runs; whiskers show the range of the two runs. Synthetic speech, 10 clips per language.

The damage was concentrated in Marathi, where the ten-language list identified the language in 2 of 10 clips and the word error rate was 77% (mean of the two runs), and in Tamil, where three clips were labelled Telugu in run 1. Output in the wrong language was often fluent text in another script, for example Telugu script for Marathi speech. The two runs gave identical text for 38 to 39 of the 40 clips in each configuration, so the difference between runs of one configuration comes from one or two clips. In informal tests with two phones, the two-language configuration also labelled some accented English speech as Hindi and wrote it in Devanagari. The clean synthetic clips did not reproduce this and we did not measure it. It is the reason the deployed configuration recognizes in the speaker's own language.

B. Translation and Script

Ten English sentences were translated into the seven Indian languages in the catalog, 70 outputs in total, and each output was translated back into English. Table IV reports the share of letters in the expected script, the chrF score of the round trip against the original sentence, and the median time of one translation request. chrF here is our own implementation (character n-grams up to six, beta 2), and a round trip is a rough check that meaning was kept, not a score against human references.

TABLE IV
TRANSLATION INTO SEVEN INDIAN LANGUAGES, 10 SENTENCES EACH

Target language	Native script %	Round-trip chrF	Median ms
Hindi	100	88.9	231
Tamil	100	86.6	196
Telugu	100	75.3	315
Malayalam	100	84.5	186
Gujarati	100	92.6	189
Marathi	100	87.1	175
Bengali	100	86.5	195

Every output was entirely in the expected script, so the Latin-letter retry (Section V-C) never fired: 0 of 140 outputs counting both variants were Latin-heavy, with the retry on and off. We also sent 8 romanized Hindi sentences such as "mera naam Sharad hai" with an unknown source language. The translator returned Devanagari for all of them, again without the retry. The retry is therefore a safeguard that was not exercised here, and these results give no evidence that it improves anything. Telugu had the lowest round-trip score (75.3) and Gujarati the highest (92.6). The median translation

request took 191 ms, more than twice the 80 ms in Table I, and the 95th percentile was 499 ms.

C. End-to-End Latency

To measure the whole pipeline, each test clip was sent to a media leg as 20 ms uplink frames at real-time speed, with a second leg on the same call receiving the translated audio, as two phones would. Four pairs were used (English to Hindi, Hindi to English, English to Tamil, Marathi to English), with six clips per pair for 24 calls run one at a time, and 12 more calls run three at a time. Times are taken at the media sockets and are measured from the end of speech in each clip, found as the last 20 ms frame above 3% of the peak level. They exclude the phone's capture and playback and the uplink and downlink between phone and server, so they are not mouth-to-ear times.

TABLE V
MEASURED STAGE TIMES AFTER THE END OF SPEECH, AGAINST THE BUDGET OF TABLE I

Stage	Budget ms	Median ms (24 calls)	95th pct. ms	Median ms (3 at a time, 12 calls)
End of speech to recognition final	330	396	718	406
Recognition final to translation done	80	194	811	174
Translation done to first synthesized byte	150	810	947	670
End of speech to first translated audio	560	1428	2160	1255

One Hindi clip contained two sentences and the first translation started before the clip ended; it is left out of the last row, which therefore has 23 calls in the one-at-a-time case. All 36 calls produced translated audio and none returned an error message. From the end of speech to the first translated audio the median was 1.43 s one at a time and 1.25 s three at a time, so running three calls together made no visible difference at this small scale. The largest part is the first synthesized byte, 0.81 s after translation finished (Fig. 3). In isolation the synthesizer returned its first chunk after 0.36 to 1.05 s (median 0.57 s, Section X-E), so part of this stage is the synthesizer itself and we did not find out where the rest goes. The translation stage has a median of 0.19 s but its 95th percentile was 0.81 s, and recognition had one slow case of 1.6 s.

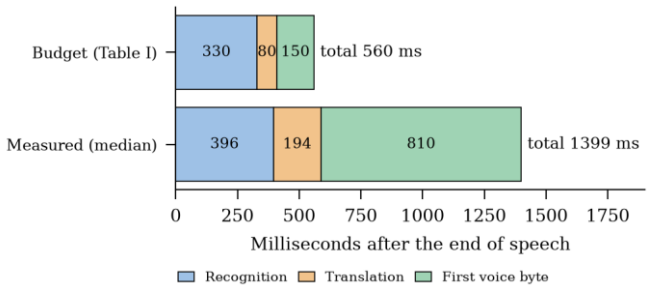


Fig. 3. Median time per stage after the end of speech, against the budget of Table I (recognition is the sum of the silence endpoint and the first usable recognition). Measured values are medians of 24 calls run one at a time.

D. Changing One's Own Language During a Call

In this test a speaker talks in one language, changes the language they speak, and talks again. Two changes were used, Hindi to English and English to Marathi, with four sentences each. After the first sentence and its translation had played, the client sent the new language and the next sentence followed either 0.3 s or 1.0 s later. In a control run the recognizer was not told about the change. The word error rate was computed on the second sentence (Table VI).

TABLE VI
RECOGNITION OF THE SECOND SENTENCE AFTER A CHANGE OF SPOKEN LANGUAGE

Condition	Calls	WER %	Exact
Language change sent 0.3 s before speech	8	4.5	6/8
Language change sent 1.0 s before speech	8	6.4	4/8
No language change sent (control)	8	100	0/8

When the recognizer was told about the change, the second sentence was recognized almost correctly even 0.3 s after the message was sent, so the restart did not cost the first words in this test. The errors we inspected were differences such as the digit 2 for the word two. Without the message the recognizer kept working in the old language and every sentence was wrong: Hindi recognition wrote English words in Devanagari, for example the text of "we booked two tickets for the evening train", and English recognition wrote Marathi words in Latin letters. The test used only 24 calls and synthetic speech, so it shows that the mechanism works, not how often it fails for real speakers.

E. Echo Guard

The echo gate was checked by replaying real synthesis timings. For six sentences (English, Hindi and Tamil, 3.2 to 5.6 s of speech), we recorded when each audio chunk arrived from the synthesizer and computed how much of the playback on the listener's phone was covered by the gate. Playback was assumed to start when the first chunk arrived and to continue without gaps. Table VII compares the first gate (the latest chunk's duration plus 0.35 s from its arrival) with the gate that follows the playback queue. This is a calculation on measured timings. We did not measure sound leaking from a speaker into a microphone.

TABLE VII
SHARE OF PLAYBACK COVERED BY THE ECHO GATE, FROM MEASURED CHUNK ARRIVAL TIMES

Sentence	Audio s	Chunks	Speed, times real time	Per-chunk gate %	Queue gate %
English 1	4.3	214	31	12	100
English 3	3.6	183	21	15	100
English 7	3.2	160	25	16	100
Hindi 1	5.6	281	32	10	100
Hindi 3	4.1	206	28	13	100
Tamil 3	3.4	171	16	17	100

Speech arrived 16 to 32 times faster than it plays, and all chunks of a sentence arrived within 0.2 s of the first. The per-chunk gate was therefore over long before the voice had finished: it covered only 10 to 17% of the playback and left 2.7 to 5.1 s of each sentence unguarded (Fig. 4). The queue-length gate covers all of it by construction. It stays correct only while the phone plays the queued audio without gaps; if the phone's buffer runs dry or playback is cancelled, the gate

and the real playback can disagree, which is why a cancellation resets it. A gate that covers the whole playback also means that a person who starts speaking during the translation is not heard, which Section VII describes as a cost.

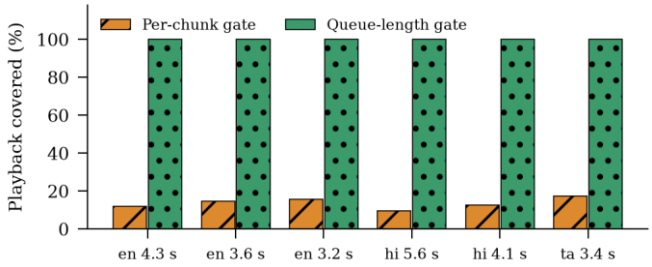


Fig. 4. Share of translated playback covered by the echo gate for six sentences, per-chunk gate against queue-length gate, from measured chunk arrival times.

F. Implementation Check: Candidate Lists

The three subsections that follow keep the function-level checks of the earlier version of this paper. They show what the code returns, not how well the system performs. Table VIII shows what lid_candidates returned for five sets of call preferences. In every row the preferred languages came first. Four base languages were left out each time, and those four cannot be detected on that call. With no preferences the list follows catalog order and drops French, Spanish, Dutch and Portuguese. German ends up 9th and Chinese 10th.

TABLE VIII
CANDIDATE LISTS RETURNED BY LID_CANDIDATES. FOURTEEN BASE LANGUAGES ARE CONFIGURED AND TEN ARE RETURNED.

Call preferences	Ten locales returned, in order	Absent, so not detected
None (catalog order)	hi-IN, ta-IN, te-IN, ml-IN, gu-IN, mr-IN, bn-IN, en-IN, de-DE, zh-CN	fr-FR, es-ES, nl-NL, pt-PT
Tamil, Hindi	ta-IN, hi-IN, te-IN, ml-IN, gu-IN, mr-IN, bn-IN, en-IN, de-DE, zh-CN	fr-FR, es-ES, nl-NL, pt-PT
French, Spanish	fr-FR, es-ES, hi-IN, ta-IN, te-IN, ml-IN, gu-IN, mr-IN, bn-IN, en-IN	de-DE, zh-CN, nl-NL, pt-PT
Portuguese (Portugal and Brazil)	pt-PT, hi-IN, ta-IN, te-IN, ml-IN, gu-IN, mr-IN, bn-IN, en-IN, de-DE	zh-CN, fr-FR, es-ES, nl-NL
English (India), German	en-IN, de-DE, hi-IN, ta-IN, te-IN, ml-IN, gu-IN, mr-IN, bn-IN, zh-CN	fr-FR, es-ES, nl-NL, pt-PT

Fig. 5 puts that run next to a call where French and Spanish are preferred. French moved to rank 1 and Spanish to rank 2, and German and Chinese were pushed out. Dutch and Portuguese stayed out in both runs. When we asked for both Portuguese locales, only pt-PT was kept. pt-BR was dropped because it shares the base language.

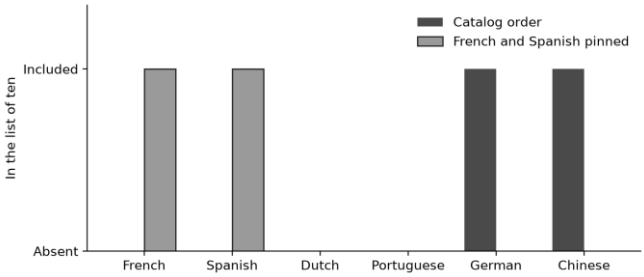


Fig. 5. Membership of six base languages in the ten-locale list, under catalog order and with French and Spanish pinned. A bar at Included means lid_candidates returned that language.

We also called lid_candidates once for each of the 15 catalog rows on its own. The preferred locale was first in all 15 lists. For the same rows, script_rule gave a native-script instruction 7 times, for the seven Indic languages, and an empty one for the other 8.

So the cap has a real cost. If neither person on a call prefers French, Spanish, Dutch or Portuguese, a switch into one of them is not detected at all. Pinning the two preferred languages first keeps the languages that matter most for that call inside the list. Section X-A shows that the list itself also matters: filling the unused slots with other languages cost accuracy, so keeping many languages in the list is not free either.

G. Implementation Check: Same-Language Skip

Table IX lists what same_language returned for 12 code pairs. A yes means the recognized text is spoken without going to the translator. Some pairs with different codes still matched: en-IN with en, hi-IN with hi, zh-CN with zh-Hans, and pt-PT with pt-pt. ta-IN with ta-IN matched as well.

TABLE IX
SAME-LANGUAGE DECISIONS RETURNED BY SAME_LANGUAGE. YES MEANS THE TEXT IS NOT SENT TO THE TRANSLATOR.

Spoken code	Destination code	Skip translation
en-IN	en	Yes
hi-IN	hi	Yes
zh-CN	zh-Hans	Yes
pt-PT	pt-pt	Yes
ta-IN	ta-IN	Yes
hi-IN	mr-IN	No
pt-PT	pt-BR	No
pt-BR	pt	No
de-DE	en-IN	No
auto	hi-IN	No
(empty)	ta-IN	No
und	fr-FR	No

Hindi did not match Marathi, and pt-PT did not match pt-BR. pt-BR against the plain code pt was also a no. This is because European Portuguese registered the base code pt first, so the check compares pt with pt-pt. A source of auto, empty or und never matched any destination, so in those cases the text is always sent to the translator.

H. Implementation Check: Latin Letters

Fig. 6 shows the Latin share of letters for six test strings given to is_latin_heavy. The retry only happens when the share is above one half. An English sentence, 13 of 13 letters Latin, was retried. Devanagari alone, 0 of 7, was not. The

word hello followed by Devanagari came to 5 of 9 and was retried, but four Latin letters followed by Devanagari came to 4 of 9 and was not. The other two strings, 2 of 6 and 1 of 8, were also under the line.

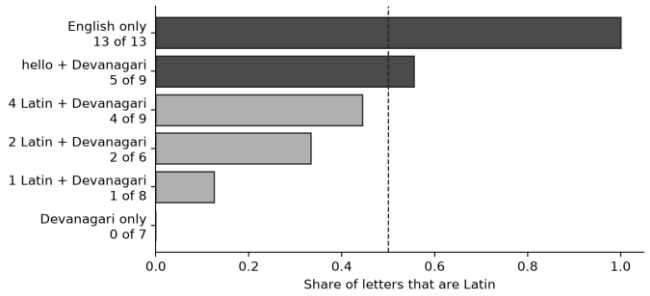


Fig. 6. Latin share of letters in six inputs to is_latin_heavy. The dashed line is one half. Inputs above the line are retried in the native script.

We picked these strings to find the threshold. They are not output from the cloud translator. The 180 ms Silero VAD [9] setting from Section VI was not re-scored on a speech corpus in this paper.

XI. Protocol Details and Remaining Work

Section X ran a reduced form of the protocol below. This section records what was run and what is left, so the measurements can be repeated and extended.

Conditions: The providers were Azure Speech and Azure Translator in Central India, called from one Windows PC that ran the orchestrator in Docker. Four recognition configurations (Table III) were run on the same 40 clips, twice. The media-socket tests used the deployed configuration: recognition in the speaker's language, no forwarding of the original voice, the queue-length echo gate, and the same-room guard off. The scripts eval_pipeline.py and analyze_eval.py in the project repository regenerate every number and figure in Section X from Azure keys and the orchestrator.

Content: Not yet done: public read-speech corpora with human reference translations (for example FLEURS [8]), real speakers, code-switching, names and numbers, a spoken language that is outside the candidate list, and a bilingual reference transcriber. The 40 synthetic clips used here cannot support claims about all Indian languages.

Metrics: Word error rate and character error rate were computed after Unicode normalization (NFC), lower-casing and removal of punctuation, with whitespace-separated words; the digit 2 against the word two counts as an error. chrF is our own implementation (n up to six, beta 2). Confidence intervals are 95% bootstrap intervals over clips (4000 resamples). Latency is measured at the media sockets from the end of speech. Not yet reported: COMET, a mouth-to-ear delay with a clock shared between phone and server, task completion and overlap measures in a conversation study, and how readable listeners find partial translations.

What would be considered a successful result? We can only claim improvement for a specific pair if it performs better without affecting performance on the speech-based conditions significantly (in terms of recognition accuracy, task completion, translation scores). We can only claim an improvement in latency when the client-side delivery is partial translation with a shorter silence threshold. We cannot claim improvement based on script accuracy alone. If there is an improvement in script accuracy at the cost of a decrease in translation score (which could still be a win for the business), it should be mentioned in the research paper.

XII. Limitations

The quality of recognition, translation, and synthesis is determined by the services configured at execution time. Provider names, regions, and dates should be fixed (frozen) in any assessment document. Simply swapping the translator for IndicTrans2 [7], or replacing the cascade system with an end-to-end model [3], [5], is not enough to fully test a session involving 'listening language'.

In the deployed configuration a person must choose the language they speak, and speech in another language is recognized in the chosen one until the setting is changed (Table VI). In the automatic configurations a language outside the candidate list is never detected. The session did not include anything that could be classified as a code-mixing model. Names and numbers are either copied as they are by the cloud translator or omitted. The echo control works as a timer with a fixed 0.35-second tail. Depending on loudspeakers, Bluetooth headsets or a large jitter buffer, acoustic echo can leak past the gate, or the gate can cut off the start of real speech.

The capture mechanism does not yet have an integrated audio device record that can be shared with the phone call stack. Even if we refer to Table I, the latency of a translated call will significantly exceed the 'one-way latency' [2] considered acceptable on the telephone; Consequently, users are likely to start speaking while the translation is in progress. Audio and copy of the microphone are routed to speech translation services configured for this specific deployment. In the case of recordings made for Section XI, it is mandatory to obtain user consent, provide information about the data retention period, and provide an option to delete the call audio. This paper does not provide such a data management plan; All that is said is that a data management plan must be in place before storing such records.

The measurements in Section X have limits. The speech is synthetic, from one voice per language, with ten sentences in four languages, so recognition scores are optimistic and the confidence intervals are wide. Translation was scored only by round trip. Everything ran on one PC with one set of cloud services in one region, with at most three calls at once, and each experiment was run once, except recognition, which was run twice. Latency was measured at the sockets and not mouth to ear, and about 0.2 s of the first-byte delay in Table V is not explained by the synthesizer's own start-up time (median 0.57 s in isolation). In informal phone tests the two-language configuration labelled some accented English speech as Hindi, which these clips did not show.

XIII. Conclusion

We started from the idea that the main requirement for translated phone calls is to let each participant choose the language they want to hear, and built the gateway around it. Each user has a preferred language, each person's speech is recognized and translated into the other person's chosen language, and the voice is synthesized in that language. Indian target languages are returned in native script, and echo is handled by a gate that follows the playback queue, together with ducking and barge-in.

The measurements changed the design. A ten-language candidate list, which the first version used, gave about four times the word error rate of a list of the two call languages (42.1–43.3% against 9.2–12.1% on clean synthetic speech), and the deployed system now recognizes in the speaker's own language. A speaker who changes language needs a recognizer restart, and in our test the restart did not lose the first words. The median time from the end of speech to the first translated audio was 1.43 s, about twice the budget, and over half of it is the wait for the first synthesized byte. The echo gate that followed individual chunks left most of each playback unguarded. Evaluation on real speech is still to be done: public corpora with human references, real speakers, a conversation study, and a measurement of mouth-to-ear delay.

Appendix A: Control Messages Used by the Session

These same messages must be followed by re-implementation. They are included to facilitate verification of session consistency within the software.

Session start. src_lang is automatic and is not used for recognition. hear_lang is the language the sender speaks and chooses to hear, and recognition runs in this language. dst_lang is the preferred language of the other party. The sample rate is 16,000.

Language updated. The media message carries either dst_lang, the other party's new language, or hear_lang, the sender's own new language. A dst_lang update cancels any synthesis in progress and does not touch the recognizer. A hear_lang update restarts the recognizer in the new language.

Delay. The server emits stage offsets relative to its probe clock—typically vad_end, stt_first_partial, stt_final, mt_done, tts_first_byte, and e2e_server_ms—at the moment the first synthesized byte is recorded. e2e_server_ms is the time of the first byte minus the start of the probe. The probe starts at the first partial result (or the voice-activity end if that comes first), so it includes the speech itself and does not measure from the end of speech. It is not a mouth-to-ear delay. Differences between stages, such as mt_done minus stt_final, are valid stage durations.

Echoes. Uplink audio from a peer is discarded before recognition while that peer's echo time limit is in the future. When audio is sent to a peer, the limit is moved to the time at which everything queued for that peer will have finished playing, plus 0.35 seconds. A cancellation sets the limit to 0.35 seconds from now.

References

[1] S. C. Levinson and F. Torreira, "Timing in turn-taking and its implications for processing models of language," Frontiers in Psychology, vol. 6, Art. no. 731, Jun. 2015, doi: 10.3389/fpsyg.2015.00731.

[2] ITU-T Recommendation G.114, "One-way transmission time," International Telecommunication Union, May 2003.

[3] Y. Jia, R. J. Weiss, F. Biadsy, W. Macherey, M. Johnson, Z. Chen, and Y. Wu, "Direct speech-to-speech translation with a sequence-to-sequence model," in Proc. Interspeech, 2019, pp. 1123–1127.

[4] Y. Jia, M. T. Ramanovich, T. Remez, and R. Pomerantz, "Translatotron 2: High-quality direct speech-to-speech translation with voice preservation," in Proc. 39th Int. Conf. Machine Learning, vol. 162, PMLR, 2022, pp. 10120–10134.

[5] Seamless Communication et al., "SeamlessM4T: Massively multilingual & multimodal machine translation," arXiv:2308.11596, 2023.

[6] H. Kim, S.-W. Son, H. Cho, H. Kim, and J. Kim, "WIGVO: Real-time bidirectional speech translation over legacy PSTN calls via dual-session echo gating," in Proc. 64th Annu. Meeting Assoc. Comput. Linguistics (Vol. 3: System Demonstrations), San Diego, CA, USA, Jul. 2026, pp. 336–344, doi: 10.18653/v1/2026.acl-demo.33.

[7] J. Gala et al., "IndicTrans2: Towards high-quality and accessible machine translation models for all 22 scheduled Indian languages," Transactions on Machine Learning Research, 2023.

[8] A. Conneau, M. Ma, S. Khanuja, Y. Zhang, V. Axelrod, S. Dalmia, J. Riesa, C. Rivera, and A. Bapna, "FLEURS: Few-shot learning evaluation of universal representations of speech," in Proc. IEEE Spoken Language Technology Workshop (SLT), 2022, pp. 798–805.

[9] Silero Team, "Silero VAD: pre-trained enterprise-grade voice activity detector," GitHub repository. [Online]. Available: https://github.com/snakers4/silero-vad (accessed Oct. 2026).

[10] Microsoft, "Implement language identification – Speech service," Microsoft Learn. [Online]. Available: https://learn.microsoft.com/en-us/azure/ai-services/speech-service/language-identification (accessed Oct. 2026).

[11] M. Popović, "chrF: character n-gram F-score for automatic MT evaluation," in Proc. 10th Workshop Statistical Machine Translation, 2015, pp. 392–395.

[12] R. Rei, C. Stewart, A. C. Farinha, and A. Lavie, "COMET: A neural framework for MT evaluation," in Proc. EMNLP, 2020, pp. 2685–2702.